Aprendizaje por Refuerzo

Predicción On Policy con Aproximaciones

Luis Daniel Hernández Molinero
Ciencia de la Computación e Inteligencia Artificial
Dept. Ingeniería de la Información y las Comunicaciones

Universidad de Murcia

Máster en Inteligencia Artificial

2025 / 2026

Índice de Contenidos

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Introducción
Escalabilidad, generalización y flexibilidad a cambio de garantías teóricas

- La función de valor aproximada se representa **no como una tabla** sino como una **forma funcional parametrizada** con un vector de pesos $\mathbf{w} \in \mathbb{R}^d$.
- Escribiremos $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$ para **el valor aproximado del estado** s dado el vector de pesos $\mathbf{w}$.
- Por ejemplo, $\hat{v}$ puede ser:
  - Una **función lineal en las características del estado**, $[f_1(s), f_2(s), \ldots, f_d(s)]$, con $\mathbf{w}$ como el vector de pesos de las características.
  - La función calculada por una **red neuronal artificial multi-capa**, con $\mathbf{w}$ como el vector de pesos de conexión en todas las capas.
  - La función computada por un **árbol de decisión**, donde $\mathbf{w}$ son todos los números que definen los puntos de división y los valores de las hojas del árbol.
- Típicamente, la **dimensionalidad** de $\mathbf{w}$ es mucho menor que el número de estados ($d \ll |S|$), y cambiar un peso afecta el valor estimado de muchos estados.

  P.e. Go: 10^{600} secuencias para llegar a una de las 10^{170} combinaciones.
- Cuando se actualiza un solo estado, el **cambio se generaliza** desde ese estado para afectar los valores de muchos otros estados.
- Tal generalización hace que el **aprendizaje** sea potencialmente **más poderoso**, pero también potencialmente **más difícil de gestionar y entender**.
- Extender el aprendizaje por refuerzo a la aproximación de funciones también lo hace **aplicable a problemas parcialmente observables**, en los cuales el estado completo no está disponible para el agente.

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Aproximación de Funciones de Valor

- Nos referimos a una **actualización individual** mediante la notación $s \mapsto u$, donde s es el estado actualizado y u es el objetivo de actualización hacia el cual se desplaza el valor estimado de s.
  - La actualización de programación dinámica es $S_t \mapsto \mathbb{E}^{\pi}[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) | S_t = s]$,
  - La actualización Monte Carlo para la predicción de valor es $S_t \mapsto G_t$.
  - La actualización TD(0) es $S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t)$
  - La actualización TD de n-pasos es $S_t \mapsto G_{t:t+n}$.
- Es natural **interpretar cada actualización como un ejemplo** del comportamiento deseado de entrada-salida de la función de valor.
  - **Hasta ahora**, la actualización para **el valor estimado** de s simplemente **se desplaza una fracción** hacia el valor objetivo u, manteniendo los demás valores estimados de los demás estados sin cambios.
  - **Ahora**, la actualización $s \mapsto u$ significa que **el valor estimado** para el estado s **debería ser lo más parecido** al valor objetivo u. Además, la actualización en s se generaliza de manera que los valores estimados de muchos otros estados también se cambiarán.
- **Aprendizaje supervisado** Los métodos de aprendizaje automático que aprenden a imitar ejemplos de entrada-salida
  - Cuando las salidas son números, como u, el proceso a menudo se llama **aproximación de funciones**
- Utilizamos métodos de **aproximación de funciones** para la predicción de valor pasando a ellos el $s \mapsto u$ de cada actualización **como un ejemplo de entrenamiento**.

Aproximación de Funciones de Valor: Restricciones

- En el aprendizaje por refuerzo es importante que **el aprendizaje debe ser on-line** mientras el agente interactúa con su entorno o con un modelo de su entorno.
  - Los métodos necesitan aprender eficientemente **a partir de datos adquiridos incrementalmente**.

- El aprendizaje por refuerzo generalmente requiere métodos de aproximación de funciones capaces de manejar **funciones objetivo no estacionarias**.
  - En métodos de control basados en GPI a menudo buscamos aprender q_π mientras π cambia.
  - Incluso si la política permanece igual, **los valores objetivo de los ejemplos de entrenamiento son no estacionarios** si son generados mediante *métodos de refuerzo* (DP y TD learning).

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

El Objetivo de Predicción ($\overline{\text{VE}}$)

- **Medida de desempeño:** evaluar la calidad de las predicciones realizadas por un modelo.
- En el **caso tabular no era necesaria** una medida de desempeño (dependencia temporal).
  - **Por ejemplo**, la actualización de forma local: $V(s) \leftarrow V(s) + \alpha[Target - V(s)]$
- En el **caso aproximado sí es necesaria** una medida de desempeño (dependencia estrutural)
  - **Por ejemplo**, si modelamos los valores con una función lineal: $V_\theta(s) = \theta_1 x_1 + \theta_2 x_2$
- Definimos el **Error Cuadrático Medio de Valor**, denotado $\overline{VE}$, como la **función objetivo a minimizar**.

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

- Especificamos una **distribución de estados** $\mu(s)$ con $\mu(s) > 0, \sum \mu(s) = 1$, representando cuánto nos importa el error en cada estado s.
- Bajo entrenamiento on-policy **a $\mu(s)$ se le llama la distribución on-policy**.
  - En tareas episódicas, $\mu(s)$ se elige como la **fracción del tiempo** que se ha pasado en s.
  - En tareas infinitas, la distribución en política $\mu(s)$ es la **distribución estacionaria** bajo π.

Cálculo de la Distribución on-policy

- Queremos calcular **la distribución on-policy**.
- En una **tarea episódica**, **la distribución on-policy** *depende* de cómo se eligen los *estados iniciales* de los episodios.
- Sea $h(s)$ la probabilidad de que un episodio comience en el estado s, y sea $\eta(s)$ el *número de pasos de tiempo pasados*, en promedio, en el estado s en un solo episodio.

$$\eta(s) = h(s) + \sum_{\bar{s}} \eta(\bar{s}) \sum_{a} \pi(a|\bar{s}) p(s|\bar{s}, a), \quad \text{para todo } s \in S.$$

- La **distribución on-policy** es entonces la fracción de tiempo pasado en cada estado normalizada para sumar uno:

$$\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')}, \quad \text{para todo } s \in S.$$

- En una **tarea no episódica** (con trayectorias infinitas) en el caso discreto, $h(s) = 0$

El Objetivo de Predicción ($\overline{\text{VE}}$)
¿Se puede alcanzar siempre?

- **Error Cuadrático Medio de Valor**, denotado $\overline{\text{VE}}$, es nuestra función objetivo.

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

- Un **objetivo ideal** en términos de $\overline{\text{VE}}$ sería encontrar un **óptimo global**.
  - Un vector de pesos $\mathbf{w}^*$ tal que $\overline{\text{VE}}(\mathbf{w}^*) \leq \overline{\text{VE}}(\mathbf{w})$ para todo $\mathbf{w}$.
- **¿Es posible alcanzar este objetivo?**
  - **A veces es posible** para aproximadores de **funciones simples**, como los lineales.
  - **Rara vez es posible** para aproximadores de **funciones complejas**, como las RNA y los árboles de decisión.
- Los *aproximadores de funciones complejas* pueden buscar converger en su lugar a un **óptimo local**.
  - Esta garantía es *típicamente lo mejor* que se puede decir para aproximadores de funciones no lineales, y **a menudo es suficiente**.
  - Sin embargo, **para muchos casos de interés en RL no hay garantía de convergencia** a un óptimo, ni siquiera dentro de una *distancia acotada* de un óptimo.
  - Algunos **métodos pueden incluso divergir**, con su $\overline{\text{VE}}$ acercándose al infinito en el límite.

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Profundizamos en ...

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Métodos del Descenso de Gradiente Estocástico

- Métodos de **Descenso de Gradiente**:
  - $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla \overline{\mathrm{VE}}(\mathbf{w}_n)$, con $\overline{\mathrm{VE}}(\mathbf{w}) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$
  - El vector de pesos es un *vector columna* con un número fijo de componentes reales: $\mathbf{w} = (w_1, w_2, \ldots, w_d)^\top$.
  - $\overline{\mathrm{VE}}$ debe ser diferenciable. $\nabla f(\mathbf{w}) \doteq \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \frac{\partial f(\mathbf{w})}{\partial w_2}, \ldots, \frac{\partial f(\mathbf{w})}{\partial w_d} \right)^\top$.
  - La distribución on-policy $\mu(s)$ **no se puede calcular**.
    - La distribución $\mu(s)$ emerge como resultado de seguir la política $\pi(a|s)$ y las transiciones del entorno $p(s'|s, a)$.
    - Cuando ejecutamos episodios de acuerdo a $\pi(a|s)$ y $p(s'|s, a)$, se genera una cadena de Markov.
    - Los estados visitados durante la ejecución reflejan la distribución $\mu(s)$ de visitas esperadas.
- Una buena estrategia en este caso es **tratar de minimizar el error en los ejemplos observados**.
- Método del **Descenso de Gradiente Estocástico** (SGD, Sthocastic-Gradient Method)
  - En cada paso, observamos un nuevo ejemplo $S_t \mapsto v_\pi(S_t)$
  - Por ahora,
    - Conocemos el **valor verdadero bajo la política** $v_\pi(S_t)$.
    - La función de valor aproximada $\hat{v}(s, \mathbf{w})$ es una **función diferenciable** respecto a $\mathbf{w}$ para todo $s \in S$.
  - **Función de actualización:**

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t - \frac{\alpha}{2} \nabla \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right]^2 \tag{9.4}$$

$$\doteq \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t), \tag{9.5}$$

  - **SGD converge** si α decrementa en el tiempo de acuerdo a las condiciones $\sum_n \alpha_n(a) = \infty$ y $\sum_n \alpha_n^2(a) < \infty$

Descenso de Gradiente de Monte Carlo

- Ahora consideramos $S_t \mapsto U_t$, **no es el valor verdadero**, $v_\pi(S_t)$
  - $U_t \approx v_\pi(S_t)$ (es ruido)
  - $U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$ (es un objetivo bootstrap que usa $\hat{v}$)
- Cambiamos la **Función de actualización** del **Método del Descenso de Gradiente Estocástico** (SGD):

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t). \tag{9.7}$$

- **Si U_t es una estimación no sesgada**, es decir, si $\mathbb{E}[U_t | S_t] = v_\pi(S_t)$ para cada t, entonces $\mathbf{w}_t$ está **garantizado que convergerá a un óptimo local** bajo las condiciones usuales de para α decreciente.

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{y} \quad \sum_{n=1}^{\infty} \alpha_n^2 < \infty$$

- La **versión de descenso de gradiente de Monte Carlo** es el Descenso del Gradiente Estocástico que usa $U_t = G_t$ para la predicción de valores de estado y se **garantiza que encontrará una solución óptima local**.
  - El **objetivo de Monte Carlo** $U_t = G_t$ es **por definición una estimación no sesgada** de $v_\pi(S_t)$.
  - Exigue la condiciones de aproximación estocástica
  - Usa la ecuación de actualización (9.7)
  - **Convergerá a una aproximación óptima local** de $v_\pi(S_t)$

Algoritmo del Gradiente (Estocástico) de Monte Carlo

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[G_t - \hat{v}(S_t,\mathbf{w})\big]\nabla\hat{v}(S_t,\mathbf{w})$

Profundizamos en ...

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Métodos de Semi-gradiente

- Tenemos que, si observamos un ejemplo $S_t \mapsto V_t$:
  - Si $V_t = v_\pi(S_t)$, el método del descenso del gradiente converge a un mínimo.
  - Si $V_t = U_t$ responde a una muestra de Monte Carlo, G_t, también converge.
  - ¿Y si $V_t = U_t$ es un objetivo calculado por bootstraping? p.e. $U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$ (DT)
- Si **el target U_t es por bootstrap**, dependerán del valor actual de pesos $\mathbf{w}_t$.
  - Retornos de n-pasos: $U_t = G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}_{t+n-1}(S_{t+n}, \mathbf{w}_{t+n-1}))$
  - Programación Dinámica: $U_t = \sum_a \pi(a|S_t) \sum_{s',r} p(s', r|S_t, a)[r + \gamma \hat{v}(s', \mathbf{w}_t)]$,

  Entonces, U_t estará **sesgado** y **no producirán un verdadero método de descenso de gradiente**.
- Los pasos clave desde (9.4) hasta (9.5) **depende de que el objetivo U_t sea independiente** de $\mathbf{w}_t$.
- Los métodos *bootstrap* **no generan** ejemplos del **gradiente verdadero**, pero incluyen una parte del gradiente (Barnard, 1993).
- Llamamos **métodos de semi-gradiente**, a aquellos que **usan** la siguiente función de actualización del *descenso del gradiente estocástico*, la (9.7), usando como objetivo U_t *un valor obtenido por bootstrap*.

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[U_t(\mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

E.d. Cuando U_t depende de unos pesos $\mathbf{w}$ que aparecen en el gradiente.

Métodos Semi-gradiente TD(0)

- Aunque los métodos de *semi-gradiente* (bootstrap) no convergen tan robustamente como los métodos de gradiente, *convergen de manera confiable* en casos importantes.

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})\big]\nabla\hat{v}(S,\mathbf{w})$
    $S \leftarrow S'$
  until S is terminal

El Algoritmo TD de Semi-gradiente con n Pasos

n-step semi-gradient TD for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameters: step size $\alpha > 0$, a positive integer n
Initialize value-function weights $\mathbf{w}$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
All store and access operations (S_t and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t+1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n \hat{v}(S_{\tau+n}, \mathbf{w})$ $(G_{\tau:\tau+n})$
  | $\mathbf{w} \leftarrow \mathbf{w} + \alpha\,[G - \hat{v}(S_\tau, \mathbf{w})]\,\nabla \hat{v}(S_\tau, \mathbf{w})$
  Until $\tau = T - 1$

$$\mathbf{w}_{t+n} \doteq \mathbf{w}_{t+n-1} + \alpha\,[G_{t:t+n} - \hat{v}(S_t, \mathbf{w}_{t+n-1})]\,\nabla \hat{v}(S_t, \mathbf{w}_{t+n-1}),$$
$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w}_{t+n-1}).$$

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Métodos Lineales

- $\hat{v}(\cdot, \mathbf{w})$ es una **función lineal** del vector de pesos w.
- Correspondiendo a cada estado s, hay un vector de valores reales $\mathbf{x}(s)$ con **el mismo número de componentes** que $\mathbf{w}$: $\mathbf{x}(s) \doteq (x_1(s), x_2(s), \ldots, x_d(s))^\top$, $\mathbf{w} \doteq (w_1, w_2, \ldots, w_d)^\top$
- Los **métodos lineales** aproximan la función de valor-estado mediante el **producto interno** entre w y $x(s)$:

$$\hat{v}(s, \mathbf{w}) \doteq \mathbf{w}^T \mathbf{x}(s) = \sum_{i=1}^{d} w_i x_i(s).$$

- El vector $\mathbf{x}(s)$ se llama **vector de características** que representa el estado s.
  - Cada componente $x_i(s)$ de $\mathbf{x}(s)$ es el valor de una función $x_i : \mathcal{S} \to \mathbb{R}$.
  - Para métodos lineales, **las características son funciones base** para el *conjunto de funciones aproximadas*.
    - Se verá algún ejemplo posteriormente.
- **El gradiente** de la función de valor aproximada con respecto a w en este caso es: $\nabla \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$
- Así, en el caso lineal, **la actualización SGD** general (9.7) se reduce a una forma particularmente simple:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[U_t - \hat{v}(S_t, \mathbf{w}_t) \right] \mathbf{x}(S_t).$$

- **El caso SGD lineal** es uno de los más favorables para el análisis matemático.
  - Los *resultados de convergencia* útiles para sistemas de aprendizaje suelen ser *para métodos de aproximación de funciones lineales (o más simples).*
  - **En el caso lineal, solo hay un óptimo.**

Profundizamos en ...

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

§Convergencia de los Métodos Lineales

Porque los estados se actualizan de acuerdo a la distribución on-policy.

- El algoritmo **Monte Carlo con gradiente** converge al **óptimo global** del VE bajo aproximación lineal de funciones si α se reduce con el tiempo de acuerdo con las condiciones habituales.
- El algoritmo **TD(0) con semi-gradiente** **también converge** bajo aproximación lineal de funciones.
  - Esto no se deduce de resultados generales sobre el descenso estocástico de gradiente (SGD).
  - El vector de pesos *converge a un punto cercano al óptimo local.*
- Para el **caso continuo**, la actualización en cada tiempo t para el algoritmo de semi-gradiente TD(0) es:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[R_{t+1} + \gamma \mathbf{w}_t^T \mathbf{x}_{t+1} - \mathbf{w}_t^T \mathbf{x}_t \right] \mathbf{x}_t$$

- Una vez que el sistema ha alcanzado el **estado estable**, para cualquier $\mathbf{w}_t$, el vector de pesos esperado puede escribirse como:

$$\mathbb{E}[\mathbf{w}_{t+1} | \mathbf{w}_t] = \mathbf{w}_t + \alpha \left(\mathbf{b} - \mathbf{A}\mathbf{w}_t \right)$$

  donde

$$\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t] \in \mathbb{R}^d, \quad \mathbf{A} \doteq \mathbb{E}\left[\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^T \right] \in \mathbb{R}^{d \times d}$$

- Si el semi-gradiente TD(0) **converge**, debe converger al **punto fijo TD** del vector de pesos $\mathbf{w}_{TD}$:

$$\mathbf{b} - \mathbf{A}\mathbf{w}_{TD} = 0 \quad \Rightarrow \quad \mathbf{b} = \mathbf{A}\mathbf{w}_{TD} \quad \Rightarrow \quad \mathbf{w}_{TD} \doteq \mathbf{A}^{-1}\mathbf{b}$$

- En el **punto fijo TD**, también se ha demostrado (en el caso continuo) que el VE está dentro de una expansión acotada del error más bajo posible: $\sqrt{VE(\mathbf{w}_{TD})} \leq \dfrac{1}{1-\gamma} \min_{\mathbf{w}} VE(\mathbf{w})$.

Profundizamos en ...

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Construcción de Características para Métodos Lineales

- El **éxito** de los métodos lineales (*LM*) **depende** críticamente de **cómo se representa los estados en términos de características**.
- **Elegir características** apropiadas para la tarea es una forma importante de **añadir conocimiento previo del dominio** a los sistemas de aprendizaje por refuerzo.
  - Intuitivamente, las características deben corresponder a los aspectos del espacio de estados a lo largo de los cuales la generalización puede ser apropiada.
- Una **limitación** de la forma lineal es que *no considera ninguna interacción entre características*
  - Ejemplo: Se tienen dos atributos:
    - $s_1 = x_1(s)$: Si hay un obstáculo cerca (1 = no, 0 = si).
    - $s_2 = x_2(s)$: Si el robot tiene un alto nivel de batería (1 = sí, 0 = no).
    - Se considera un estabo bueno cuando no hay obstáculo cerca ($s_1 = 1$) y hay suficiente batería ($s_2 = 1$). Cualquier otro se considera malo.
    - El modelo lineal $\hat{v}(s, \mathbf{w}) = w_1 s_1 + w_2 s_2$

      No captura la interacción entre s_1 y s_2. simplemente suma las contribuciones de w_1 y w_2
- Necesitamos modos de **definir características para combinaciones** de dos o más **dimensiones subyacentes del estado**.

Con más detalle ..

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Polinomios

- Aunque las **características polinomiales básicas** que discutimos aquí *no funcionan tan bien como otros tipos de características* en el aprendizaje por refuerzo, sirven como una buena introducción porque son simples y familiares.
- Supongamos un estado representativo s, con **dos componentes**, $\mathbf{x}(s) = (s_1, s_2)^T$, $s_1 \in \mathbb{R}$ y $s_2 \in \mathbb{R}$.
  - Si $s_1 = s_2 = 0$ no quiero que la función valga 0 porque considero que es un estado objetivo deseable.
  - Con solo (s_1, s_2) **no sería capaz de tener en cuenta ninguna interacción entre estas dimensiones**.
- Se puede mejorar representando s mediante el vector de características de cuatro dimensiones
  - Con $\mathbf{x}(s) = (1, s_1, s_2, s_1 s_2)^T$ podemos construir la función $\hat{v}(s, \mathbf{w}) = 1 + w_1 s_1 + w_2 s_2 + w_3 (s_1 \cdot s_2)$
    - El término 1 evita que pares (0,0) se consideren indeseables.
    - El nuevo término $(s_1 \cdot s_2)$ ajusta el valor solo cuando ambas características son 1
    - Ahora el modelo puede asignar un valor más bajo a los estados malos.
  - La característica inicial 1 permite representar funciones afines en los números de estado originales, y la característica final, $s_1 s_2$, permite tener en cuenta **interacciones**.
- Podemos usar vectores de características de dimensiones más altas.
  - Ej: $\mathbf{x}(s) = (1, s_1, s_2, s_1 s_2, s_1^2, s_2^2, s_1 s_2^2, s_1^2 s_2, s_1^2 s_2^2)^\top$
- En general, $x_i(s) = s_i = \prod_{j=1}^{k} s_j^{c_{i,j}}$, $s_j \in \mathbb{R}$, $c_{i,j} \in \{0, 1, \ldots, n\}$. En total $(n+1)^k$ características.

Con más detalle ..

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Base de Fourier

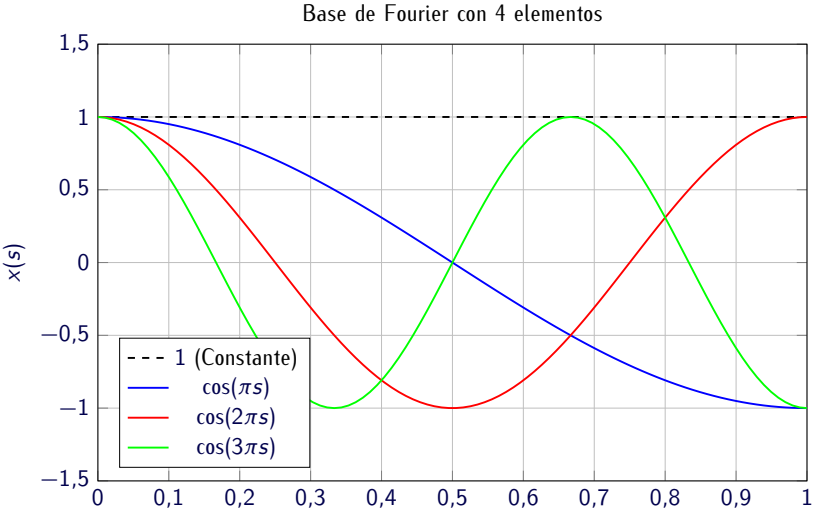
- La **base de Fourier** se utiliza **para aproximar una función objetivo** que **tiene patrones periódicos, suaves o continuos, o** cuando se necesita una representación eficiente para capturar **variaciones complejas**.
  - Su aplicación se basa en la descomposición de la función en una **combinación de senos y cosenos** con diferentes frecuencias.
- Hay varios **motivos** para usar base de Fourier y la transformada de Fourier:
  - La funciones es **periódicas**. $f(x) = f(x + \tau)$ para todo x y algún período τ.
  - Funciones definidas en **intervalos acotados** (aperiódicas).
  - Son funciones **suaves** o continuas (no necesariamente periódicas).
  - Las funciones son **desconocidas** y deben ser aproximadas basándose en datos observados.
- **Si se conoce la función a aproximar**:
  - Entonces, **los pesos de las funciones base se obtienen** mediante fórmulas simples y,
  - Con suficientes funciones base, esencialmente *cualquier función puede ser aproximada* tan precisamente como se desee.
- **En RL**, donde las *funciones a aproximar son desconocidas*, las funciones base de Fourier son de interés porque son *fáciles de usar* y pueden funcionar bien en una variedad de problemas de RL.

Construcción de la Base

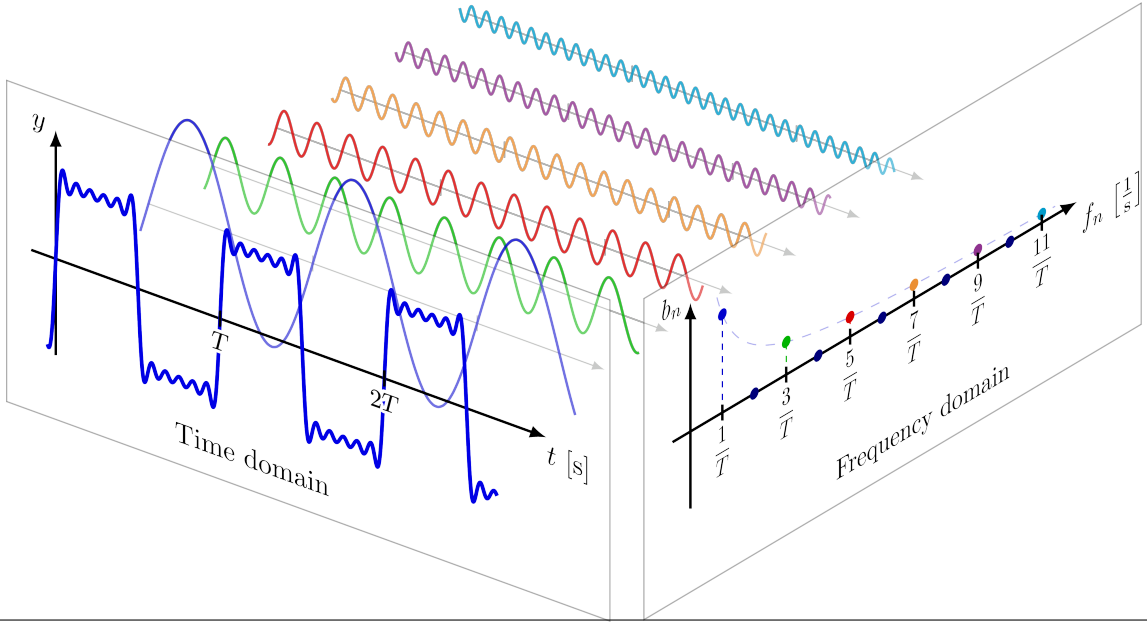
- Las **funciones periódicas** τ se pueden aproximar por una **combinación lineal de funciones seno y coseno** que sean periódicas con período τ. P.e. $cos\left(\frac{2\pi}{\tau}x\right)$ tiene periodo τ.
- Si deseamos aproximar una **función aperiódica** definida sobre un intervalo acotado, podemos usar estas **funciones base - cosenos y bases - de Fourier** estableciendo: $\tau =$ longitud del intervalo.
- Podemos usar **solo las funciones coseno**: Al establecer τ como el **doble de la longitud del intervalo** de interés y restringirnos al **medio intervalo** $[0, \tau/2]$. Esto es útil cuando la función es **par**: $f(-x) = f(x)$.
- Alternativamente, es posible usar **solo funciones seno**, cuyas combinaciones lineales son siempre funciones **impares**: $f(-x) = -f(x)$.
- No excluye usar **ambas funciones seno y coseno** para aproximar sobre el intervalo $[0, \tau/2]$, lo que podría ser **ventajoso** en algunas circunstancias.
- **Construcción de la base:**
  - Cada estado es $s = (s_1, s_2, \ldots, s_k)^\top$, con $s_i \in [0, 1]$
  - La i-ésima característica en la base coseno de Fourier de orden-n se escribi como $x_i(s) = \cos\left(\pi \mathbf{s}^\top \mathbf{c}_i\right)$, donde $\mathbf{c}_i = (c_{i1}, c_{i2}, \ldots, c_{ik})^\top$, con $c_{ij} \in \{0, 1, \ldots, n\}$ para $j = 1, \ldots, k$ y $i = 1, \ldots, (n+1)^k$.
  - Las características **pueden ser desplazadas y escaladas** para ajustarse al espacio de estados delimitado de una aplicación particular.
- **Se sugiere** establecer el parámetro de tamaño de paso para la característica x_i como: $\alpha_i = \frac{\alpha}{\sqrt{c_{i1}^2 + \cdots + c_{ik}^2}}$, excepto cuando cada $c_{ij} = 0$, en cuyo caso $\alpha_i = \alpha$.

Base de Fourier para $n = 3$ y $k = 1$

- El vector **base de Fourier** son $(n+1)^k = (3+1)^1 = 4$ elementos para:
  - **Orden** $n = 3$ (máximo valor de frecuencia en cada dimensión).
  - **Dimensión** $k = 1$ (una variable: s_1).

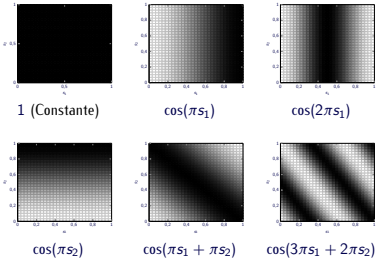


Bases de Fourier para $k = 1$



Base de Fourier para $n = 3$ y $k = 2$

- Construimos la matriz **base de Fourier** para:
  - **Orden** $n = 3$ (máximo valor de frecuencia en cada dimensión).
  - **Dimensión** $k = 2$ (dos variables: s_1 y s_2).
- La matriz representará las combinaciones de frecuencias en las dos dimensiones (s_1 y s_2) usando términos de la forma: $[\cos(\pi(c_1 s_1 + c_2 s_2))$, donde c_1, c_2 son los coeficientes de frecuencia en cada dimensión y varían entre **0 y 3** porque $n = 3$.



- La matriz está organizada como:
  1. 1ª fila ($c_2 = 0$): Frecuencias en s_1 ($c_1 = 0, 1, 2, 3$).
  2. 2ª fila ($c_2 = 1$): Combinaciones cruzadas entre s_1 y s_2 con $c_2 = 1$.
  3. 3ª fila ($c_2 = 2$): Combinaciones cruzadas con $c_2 = 2$.
  4. 4ª fila ($c_2 = 3$): Combinaciones cruzadas con $c_2 = 3$.
- **Número total de términos:** $(n+1)^k = (3+1)^2 = 16$
- Los 16 términos de la base son:

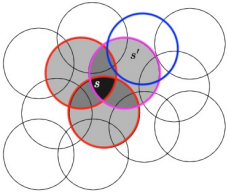
$$\mathbf{x}(s) = \begin{bmatrix} 1 & \cos(\pi s_1) & \cos(2\pi s_1) & \cos(3\pi s_1) \\ \cos(\pi s_2) & \cos(\pi s_1 + \pi s_2) & \cos(\pi s_1 + 2\pi s_2) & \cos(\pi s_1 + 3\pi s_2) \\ \cos(2\pi s_1) & \cos(2\pi s_1 + \pi s_2) & \cos(2\pi s_1 + 2\pi s_2) & \cos(2\pi s_1 + 3\pi s_2) \\ \cos(3\pi s_1) & \cos(3\pi s_1 + \pi s_2) & \cos(3\pi s_1 + 2\pi s_2) & \cos(3\pi s_1 + 3\pi s_2) \end{bmatrix}.$$

- Para la actualización se proporcionan todas las filas consecutivas: un vector fila.

Con más detalle ..

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

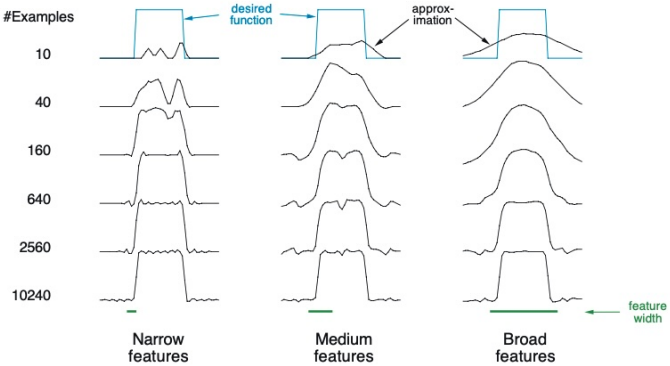
Codificación Gruesa (Coarse Coding)



- Consideremos una tarea en la que la representación natural del conjunto de estados es un **espacio continuo bidimensional**.
  - Las características corresponden a círculos en el espacio de estados.
  - Si el estado está dentro de un círculo, entonces la característica correspondiente tiene el valor 1 y se dice que está **presente**; de lo contrario, la característica es 0 y se dice que está **ausente**.
- Este tipo de característica con valores **1-0** se llama una **característica binaria**.
- Representar un estado con características que se superponen de esta manera (aunque no necesitan ser círculos o binarias) se conoce como **codificación gruesa**.
- Suponiendo una aproximación de función lineal mediante descenso de gradiente, consideremos el **efecto del tamaño y la densidad** de los círculos.
  - Cada círculo corresponde a un peso (una *componente individual* de w)
  - Si entrenamos en un estado, un punto en el espacio, entonces **los pesos de todos los círculos que intersectan ese estado se verán afectados**.
  - P.e. Cuado se actualice los pesos para s del gráfico, afectará a 3 pesos.
- La función de valor aproximada solo sumará si $x_i \neq 0$: $\hat{v}(s, w) \doteq w^T x(s) \doteq \sum_{i=1}^{d} w_i x_i(s)$

Ejemplo: Grado de Grueso en Codificación

- Aproximación de función lineal basada en codificación gruesa para aprender una función de onda cuadrada.
- Con sólo **una dimensión**, las regiones son *intervalos* en lugar de círculos.
- El aprendizaje se repitió con **tres tamaños diferentes de intervalos**: estrecho, medio y amplio.
  - Los tres casos tenían la *misma densidad de características*, alrededor de 50 en el rango de la función que se aprende.
- Los ejemplos de entrenamiento se **generaron uniformemente al azar** sobre este rango.

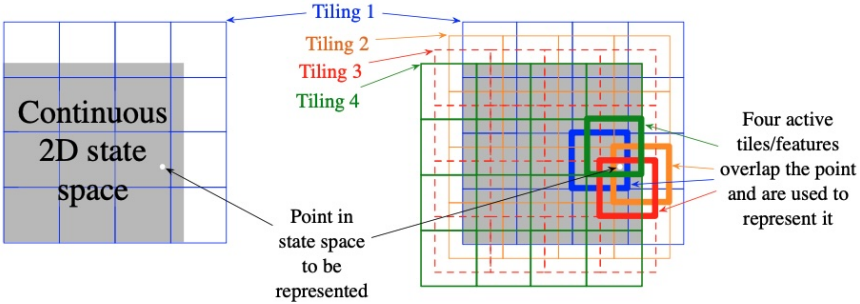


Con más detalle ..

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Codificación por Teselados (Tile Coding)

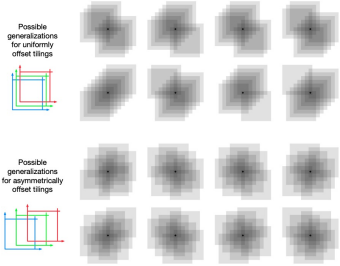
- **Teselado:** una partición del espacio en teselas.
- **Si solo se usa una partición,** solo generaliza a los estados que caen en teselas previamente entrenadas.
- **Se superponen** varios teselados (uno por característica) **y se desplazan** una freacción de la tesela.
- El vector de características $\mathbf{x}(s)$ tendrá **una componente para cada tesela** en cada teselado.
- *Ejemplo:*



- En este ejemplo, hay $4 \times 4 \times 4 = 64$ componentes, todos los cuales serán 0 excepto los cuatro correspondientes a las 'tiles' en las que cae s.

Codificación por Teselados (Tile Coding)

- Si el número, n, de características activas, cumple $n \ll d$: Hay ventajas computacionales (solo sumas)
  - *Ejemplo:* Un espacio de estados dividido en 50 mosaicos/teselas. Un estado s cumple $\mathbf{x}(s) = [1, 0, 1, 0, 0, \ldots, 0]$. La aproximación de la función de valor $\hat{v}(s)$ es $\hat{v}(s) = w_1 + w_3$
- **La generalización** (capacidad del modelo para estimar valores de cualquier estado)**se ve afectada** por:
  - **El desplazamiento:**
    Las teselaciones desplazadas asimétricamente, la generalización es más esférica y homogénea.



  - **El patrón:**
    Las teselas cuadradas generalizan aproximadamente igual en cada dimensión. Las alargadas en una dimensión. Si unos son más delgados discriminan más. Si en diagonal, generalizan a lo largo de la diagonal. También pueden ser irregulares (aunque raros en la práctica).

- El número de teselados, tamaño de las teselas, desplazamiento, patrón, determinarán la resolución o finura de la aproximación asintótica.

Con más detalle ..

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

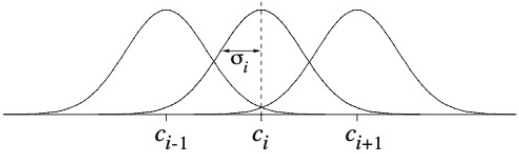
Funciones de Base Radial (RBF)

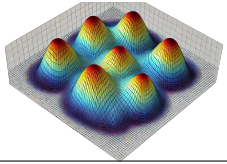
- Las **funciones de base radial (RBFs)** son la *generalización natural de la codificación gruesa* a *características de valores continuos*.
- En lugar de que cada característica sea 0 o 1, **puede ser cualquier valor en el intervalo [0, 1]**, reflejando varios grados en los que la característica está presente.
- Una característica RBF típica, x_i, es la **respuesta *Gaussiana*** $x_i(s)$

$$x_i(s) = \exp\left(-\frac{\|s - c_i\|^2}{2\sigma_i^2}\right)$$

(De)crecimiento exponencial en función de la distancia de un estado a un prototipo de la característica en relación con el *ancho* de la característica:

- s: Estado actual.
- c_i: Centro prototípico de la función base radial i.
- σ_i: Parámetro de ancho, controla la dispersión o escala de la función.
- $\|s - c_i\|$: Distancia Euclidiana entre el estado s y el centro (se puede considerar otras).





Más sobre Funciones de Base Radial (RBF)

- **Campos de aplicación:** Redes neuronales, aprendizaje por refuerzo, métodos de aproximación funcional, clasificación, clustering, modelos de regresión, procesamiento de señales e imágenes, modelado de superficies y campos de potencial, control adaptativo, sistemas dinámicos, métodos de optimización, procesos Gaussianos y aprendizaje Bayesiano.
- Hay **varias funciones** de base radial:
  - *Gaussiana:* $\varphi(r) = \exp(-\gamma r^2)$
  - *Multicuadrática:* $\varphi(r) = \sqrt{1 + (\gamma r)^2}$
  - *Inversa cuadrática:* $\varphi(r) = \frac{1}{1+(\gamma r)^2}$
  - *Inversa multicuadrática:* $\varphi(r) = \frac{1}{\sqrt{1+(\gamma r)^2}}$

  donde $r = ||\mathbf{x} - \mathbf{c}||$ y $\gamma > 0$ es un parámetro de escala.
- La **principal ventaja** de las RBFs sobre características binarias es que producen *funciones aproximadas que varían suavemente y son diferenciables.*
- Aunque esto es atractivo,
  - en la mayoría de los casos **no tienen una importancia práctica significativa**.
  - Todos estos métodos **requieren** una *complejidad computacional adicional sustancial* (en comparación con la codificación gruesa) y con frecuencia *reducen el rendimiento* cuando hay más de dos dimensiones de estado.

Desarrollo

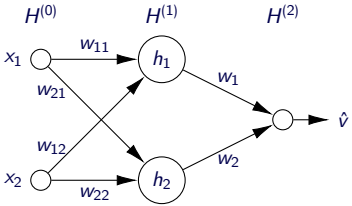
1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Redes Neuronales Artificiales como Funciones no-Lineales

- Las **redes neuronales artificiales** (**RNAs**) son ampliamente utilizadas para la aproximación de funciones no lineales.
- **Propiedad de Aproximación Universal:**
  - **Una RNA con una sola capa oculta** que contenga un número suficientemente grande de unidades sigmoides **puede aproximar cualquier función continua** en una región compacta del espacio de entrada de la red **con cualquier grado de precisión** (Cybenko, 1989).
    - Esto también es cierto para otras funciones de activación no lineales que cumplen condiciones suaves, pero la no linealidad es esencial.
- El resultado de un nodo (neurona) : $f(z) = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$
  - Entradas: $x_1, x_2, \ldots, x_n$ - Pesos asociados: $w_1, w_2, \ldots, w_n$ - Sesgo (bias): b - Función de activación: f
- **Funciones de activación** para conseguir la no-linealidad.
  - **Sigmoide:** $\sigma(x) = \dfrac{1}{1+e^{-x}}$
  - **Tangente Hiperbólica (Tanh):** $\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$
  - **ReLU (Rectified Linear Unit):** $f(x) = \text{máx}(0, x)$
  - **Softmax:** $\text{Softmax}(x_i) = \dfrac{e^{x_i}}{\sum_j e^{x_j}}$
  - **Leaky ReLU:** $f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0 \end{cases}$ donde α es un valor pequeño (p.e, 0.01).
  - **ELU (Exponential Linear Unit):** $f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha(e^x - 1) & \text{si } x \leq 0 \end{cases}$

Qué Aportan las Redes Neuronales Artificiales en RL

- **Un ejemplo** de Red Neuronal Artificial *Feedforward* (no recurrente):



  - Primera Capa (oculta)
    - $h_1^{(1)} = f_1(w_{11}x_1 + w_{12}x_2 + b_1)$
    - $h_2^{(1)} = f_2(w_{21}x_1 + w_{22}x_2 + b_2)$
  - Segunda Capa (salida)
    - $\hat{v} = h^{(2)} = f(w_1 h_1 + w_2 h_2 + b)$

- **Entrenar las capas ocultas** de una RNA es una **forma de crear automáticamente características** apropiadas para un problema dado, permitiendo representaciones jerárquicas sin depender exclusivamente de características diseñadas manualmente.
- En Aprendizaje por Refuerzo (RL), **las RNAs se pueden usar para**
  - **aprender** funciones de valor V o Q (en lo que estamos),
  - pueden **maximizar** la recompensa esperada (REINFORCE)
  - puede aplicarse en un algoritmo de **gradiente de políticas** (Actor–Crítico).

Esto va de Derivadas Parciales

- **Recordemos** la actualización de los pesos para estimar V por TD:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[\overbrace{\underbrace{R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)}_{U_t} - \hat{v}(S_t, \mathbf{w}_t)}^{Error} \right] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

- **Es necesario estimar** la **derivada parcial de una función objetivo con respecto a cada peso**, dados los valores actuales de todos los pesos de la red.
- **La forma más exitosa** de hacer esto en RNAs con capas ocultas es el **algoritmo de retropropagación (backpropagation)**.
  - Producir **buenos resultados** para redes poco profundas
  - **Puede no funcionar bien** para RNAs **con muchas capas profundas**
    - Problemas de **sobreajuste**.
    - El *backpropagation* es extremadamente **lento**.
    - Las **derivadas parciales** calculadas son casi **nulas** en las primeras capas.
- A pesar de todo, **un tipo de RNA profunda**, la **red convolucional profunda**, ha demostrado tener aplicaciones impresionantes de aprendizaje por refuerzo
  - Compensa más las ventajas que los inconvenientes.

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Aproximación de Funciones Basada en Memoria

- **Aprendizaje paramétrico:** $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$
  1. Se genera una **actualización individual**, mediante la notación $S_t \mapsto U_t$,
  2. Usamos **Descenso del Gradiente Estocástico**: $\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\left[U_t - \hat{v}(S_t, \mathbf{w}_t)\right] \nabla \hat{v}(S_t, \mathbf{w}_t)$
  3. **Tiramos la muestra** y volvemos a 1.

  Cuando se necesita un valor aproximado para un estado la función se evalúa en ese estado usando **los parámetros más recientes** producidos por el algoritmo de aprendizaje.

- **Aprendizaje perezoso (Lazy learning):** Almacena un subconjunto de ejemplos de entrenamiento en memoria a medida que llegan, sin actualizar ningún parámetro.
  - Siempre que se necesite una estimación del valor para un estado consultado, se recupera un **conjunto de ejemplos de la memoria, $\mathcal{D}$, y se utiliza** para calcular una estimación del valor para el estado consultado.
- Los métodos de aproximación de funciones basadas en memoria son **métodos no paramétricos**.
  - La **forma de la función de aproximación no está limitada a una clase parametrizada fija** de funciones.
  - La forma está **determinada por los propios ejemplos de entrenamiento**, que dependerá de cómo combinarlos.
  - Se espera que a más ejemplos de entrenamiento en la memoria, se produzcan aproximaciones más precisas para **cualquier función objetivo**.

Ejemplos de Aproximación Basada en Memoria

- **Vecino más cercano:**
  - El estado de consulta es s
  - $s^* \rightarrow g$ es el ejemplo en memoria cuyo estado s^* es el más cercano a s
  - Retornar $\hat{v}(s) = v(s^*)$
- **k-vecinos más cercanos:** $\hat{v}(s, \mathcal{D}) = \dfrac{\sum_{i \in \mathcal{N}(s)} w_i V(S_i)}{\sum_{i \in \mathcal{N}(s)} w_i}$
  - $\mathcal{N}(s)$: Conjunto de los k vecinos más cercanos al estado s.
  - $V(S_i)$: Valor almacenado para el vecino S_i.
  - w_i: Peso asignado al vecino S_i, definido como: $w_i = \frac{1}{d(s,S_i)+\epsilon}$. $d(s, S_i)$ es la distancia entre estados y $\epsilon > 0$.
- **Regresión con núcleos (Kernel regression):** Es un método basado en memoria que calcula un promedio ponderado mediante un núcleo de los objetivos de todos los ejemplos almacenados en memoria, asignando el resultado al estado consultado. $\hat{v}(s, \mathcal{D}) = \sum_{s' \in \mathcal{D}} k(s, s') g(s')$. $g(s')$ denota el *objetivo para el estado* s'.

- *Núcleo Lineal:* $k(s, s') = s^T s'$
- *Núcleo Polinómico:* $k(s, s') = (s^T s' + c)^d$
- *Núcleo Gaussiano (RBF - Radial Basis Function):* $k(s, s') = \exp\left(-\frac{s-s'^2}{2\sigma^2}\right)$

- *Objetivo TD(0):* $g(s') = r(s', a) + \gamma \hat{v}(s'')$
- *Objetivo Monte Carlo:* $g(s') = G(s') = \sum_{t=0}^{T} \gamma^t r_t$
- *Promedio Empírico (Offline):* $g(s') = \frac{1}{n} \sum_{i=1}^{n} r_i(s')$ (Usa los valores observados en $\mathcal{D}$ para calcular el promedio de recompensas asociadas al estado)

Desarrollo

1. Introducción
2. Aproximación de Funciones de Valor
3. El Objetivo de Predicción
4. Métodos de Gradiente Estocástico y Semi-gradiente
   4.1. Métodos de Gradiente Estocástico
   4.2. Métodos de Semi-gradiente
5. Métodos Lineales
   5.1. §Convergencia de los Métodos Lineales
   5.2. Construcción de Características para Métodos Lineales
      Polinomios
      Base de Fourier
      Codificación Gruesa
      Codificación por Teselados
      Funciones de Base Radial
6. Métodos No Lineales
7. Métodos No Paramétricos
8. Referencias

Referencias

- Barnard, E. (1993).Temporal-difference methods and Markov models. *IEEE Transactions on Systems, Man, and Cybernetics*, *23*(2), 357-365. https://doi.org/10.1109/21.222714
- Cybenko, G. (1989).Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, *2*(4), 303-314.
- Silver, D. (2015). Lectures on Reinforcement Learning. https://www.davidsilver.uk/teaching/
- Sutton, R., & Barto, A. (2018). *Reinforcement Learning, second edition: An Introduction*. MIT Press. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf